

Software Requirements Specification

UML System Design

Interview Prep Edition

GIC Citizen Assistant

A Multi-Tenant Retrieval-Augmented Generation (RAG) Chatbot Framework
Applied to Sri Lanka's Government Information Centre (gic.gov.lk)

This document explains what the system does, how it is built, which techniques it uses and why, and gives a full set of UML diagrams describing its structure and behaviour — written so it can be read by both a technical interviewer and a non-technical reviewer.

Prepared: July 2026 | Version 1.0

Table of Contents

1. Introduction

Purpose, problem statement, who this is for

2. System Overview in Plain English

The two halves of the system, explained with an analogy

3. Techniques & Technologies Used

RAG, embeddings, hybrid search, reranking, agentic tool-calling, multilingual pipeline, security

4. Software Requirements Specification

Scope, user classes, functional & non-functional requirements, API surface

5. UML Diagrams

Use Case, Class, Sequence, Activity, Component, Deployment, ER diagrams

6. End-to-End System Flow

Step-by-step walkthrough of a real chat request

7. Case Study — Ingesting gic.gov.lk

How the knowledge base was actually built, and bugs fixed along the way

8. Interview Q&A Cheat Sheet

Likely questions with concise, confident answers

9. Glossary of Terms

1. Introduction

1.1 Purpose

This document describes a chatbot system built to answer citizen questions about Sri Lanka's Government Information Centre (GIC) — services, government organizations, downloadable forms, and news — automatically, accurately, and in the citizen's own language (English, Sinhala, or Tamil). It also describes the underlying **framework** the chatbot is built on, which is generic enough to be reused for any other organization's knowledge base (a bank, a university, a telecom operator, etc.) simply by feeding it different documents.

In one sentence: the system reads an organization's real documents once, stores their *meaning* in a searchable form, and then — every time someone asks a question — finds the most relevant pieces of real content and asks a language model to write an answer *using only those pieces*, so the bot never has to guess.

1.2 Problem Statement

GIC's information is spread across hundreds of service pages, hundreds of organization pages, dozens of downloadable PDF forms, and news articles — some of it structured (JSON API data) and some of it just rendered as plain HTML text on a page with no API behind it. A citizen who wants to know, say, "how do I get a copy of my birth certificate" has to find the right page themselves. This system removes that burden: it ingests *all* of that content ahead of time, and answers the question directly, with a citation link back to the exact official page.

1.3 Why Not Just Use a Plain LLM (like ChatGPT) Directly?

A general-purpose LLM was not trained on GIC's specific, current data and will either say "I don't know" or — worse — confidently invent a plausible-sounding but wrong fee, form name, or phone number (this is called **hallucination**). This system instead *grounds* every answer in real, retrieved GIC content, and can also refuse to answer if nothing relevant is found in the knowledge base, rather than making something up.

1.4 Intended Audience of This Document

- **You, in an interview** — to explain the design confidently, at any depth the interviewer wants to go.
- A technical reviewer — full architecture, diagrams, and API surface.
- A non-technical stakeholder — the green "plain English" boxes throughout this document exist for exactly this purpose.

2. System Overview in Plain English

Analogy: think of the system as a **librarian who has read every single document in the building**, and who only ever answers your question by first walking to the exact shelf, pulling out the most relevant page, reading it, and then answering *from that page* — never from memory. If nothing relevant exists on the shelves, the librarian says so instead of guessing.

The system has two separate halves that run at different times:

Half 1 — Building the Library (offline, done once / whenever content changes)

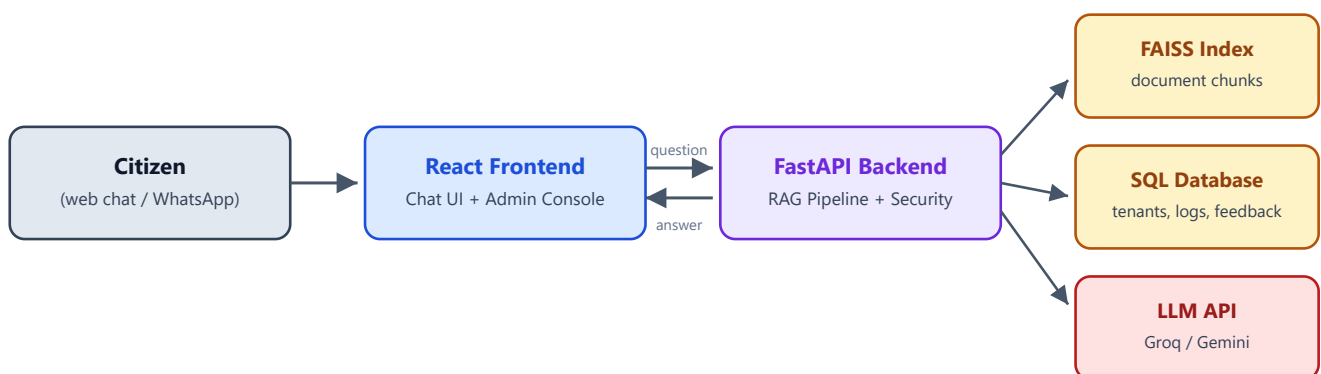
1. Fetch all the raw content (JSON APIs, web pages, PDF forms)
2. Clean and organize it into consistent records
3. Break each record into bite-sized, meaningful chunks
4. Convert each chunk into a list of numbers that captures its *meaning* (an "embedding")
5. Store all of those number-lists in a fast searchable index

Half 2 — Answering a Question (online, every single chat message)

1. Citizen types a question
2. System converts the question into the same kind of number-list
3. It searches the index for the chunks whose meaning is closest to the question
4. It double-checks and re-ranks those chunks for real relevance
5. It hands the best chunks + the question to the language model and says "answer using *only* this"
6. The answer is shown to the citizen with links back to the official source pages

This pattern — search for relevant real content first, then generate an answer grounded in it — is called **Retrieval-Augmented Generation**, or **RAG**. It is the core technique behind the entire system, covered in depth in Section 3.

2.1 High-Level Picture



One backend serves many isolated "tenants" — e.g. `client_gic`, `client_bank`, `client_university` — each with its own index & persona.

Figure 2.1 — System context: who talks to what.

3. Techniques & Technologies Used

This is the section most likely to be probed deeply in an interview. Each technique below is explained in plain English first, then with the technical detail of how it is actually implemented in this system.

3.1 Retrieval-Augmented Generation (RAG)

Plain English: instead of asking the AI to answer from memory, first go find the real, relevant paragraphs, then ask the AI to summarize/answer *using only those paragraphs*. It's "open-book exam" instead of "closed-book exam."

Technical: implemented via LangChain's tool-calling interface. The LLM is given one tool, `search_knowledge_base`, and decides for itself whether the user's message needs a knowledge-base lookup or is just small talk. When it calls the tool, the tool runs the full retrieval pipeline (Sections 3.3–3.6) and returns the top matching chunks as the tool's result, which the model then reads before writing its final answer. This is called **agentic RAG** — the model actively decides *when* to retrieve, rather than every message being forced through retrieval.

3.2 Embeddings & Vector Search

Plain English: every chunk of text is converted into a list of 384 numbers that represents its *meaning*, not its exact wording. Two sentences that mean the same thing end up as two number-lists that are close together in space, even if they don't share a single word (e.g. "how much does it cost" and "what is the fee").

Technical: model `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` — a multilingual sentence-transformer producing 384-dimensional vectors, chosen because it handles English, Sinhala, and Tamil in one shared vector space. Vectors are stored in **FAISS** (Facebook AI Similarity Search) using a flat, exact **L2 (Euclidean) distance** index — one FAISS collection per tenant, named `client_<slug>`. "Flat" means it checks every vector exactly (no approximation) — fine at this data scale (thousands of chunks); at millions of chunks you'd switch to an approximate index type like IVF or HNSW for speed (see Section 8, scaling question).

3.3 Chunking Strategy

Plain English: a whole 5-page PDF form isn't one useful search result — it's too big and vague. So every document is split into small, focused pieces before being indexed, roughly the size of one topic or one FAQ answer.

Technical: two chunking strategies depending on source shape:

- **Unstructured text/PDF** — a heading/topic/FAQ-aware chunker that respects natural document structure (headings, lists, tables extracted via `pdfplumber` for form PDFs) rather than blindly cutting every N characters.

- **Structured JSON** (the GIC REST APIs) — one chunk *per record* (per service, per organization, per news article), with explicit control over which fields become searchable **text** vs. which stay as filterable **metadata** (e.g. the citation URL is metadata, not embedded text).

Every chunk also gets its **title prefixed into the embedded text itself** (e.g. "  Obtain a Certified Copy of the Certificate of Birth: ..."), so the title contributes to the semantic match, not just the body.

3.4 Hybrid Search — Vector + Keyword

Plain English: meaning-based search is powerful but sometimes misses exact terms (like an exact form code, "Form A5", or an exact place name). Classic keyword search catches exact matches but misses paraphrases. So the system runs *both* and blends the results.

Technical: **FAISS vector search** (semantic) runs alongside a **BM25** keyword index (`rank_bm25` , the same statistical ranking family used by classic search engines like Elasticsearch). Results from both are merged using a weighted score combination (70% vector / 30% keyword by default) *plus* a **Reciprocal Rank Fusion (RRF)** bonus — a rank-based fusion formula, $\text{score} = \sum 1 / (60 + \text{rank})$, that rewards documents ranked highly by *either* method without needing the two score scales to be directly comparable.

3.5 Cross-Encoder Re-Ranking

Plain English: the first search pass is a fast but slightly rough filter — like skimming 50 book indexes quickly. Re-ranking is a slower, much more careful second pass — like actually reading the top candidates side-by-side against the question and deciding which ones truly answer it best.

Technical: the top ~50 hybrid-search candidates are re-scored by a **cross-encoder** (`cross-encoder/mmarco-mMiniLMv2-L12-H384-v1` , multilingual). Unlike the embedding model (which encodes the query and each document *separately*, a "bi-encoder"), a cross-encoder reads the **query and document together** in a single pass, which is far more accurate at judging true relevance — but too slow to run against every document in the collection, so it only ever runs on the short-list that survived hybrid search. This two-stage "fast recall, then slow precision" pattern is standard in production search/RAG systems.

3.6 Title-Match Guarantee (a targeted fix built for this project)

Plain English: real-world content sites often have several different pages with very similar titles about very similar things (e.g. "get a copy of an existing birth certificate" vs. "register a new birth"). A long, keyword-heavy page can sometimes out-score a short, precisely-titled page during re-ranking, even though the short page is the actually correct answer.

Technical: a lightweight, stopword-filtered word-overlap score is computed between the user's query and each candidate's own **title** / **name** metadata field. If any candidate's title overlaps the query above a 0.6 threshold, that candidate is **guaranteed a slot** in the final result set — even if hybrid search or the cross-encoder scored its body text lower — by being prepended and bumping the weakest surviving slot. This is a deliberately retrieval-level fix (filtering on the label the source itself assigned), not a prompt-level disclaimer, so the correct source is what gets cited, not just caveated.

3.7 Query Understanding — Normalization, Rewriting, HyDE, Multi-Query Fusion

Plain English: people don't type perfect search queries — they type "how 2 get birth cert", or type in Sinhala using English letters ("Singlish"), or ask a vague follow-up like "and if I was born in Badulla?". Before searching, the raw message is cleaned up into something that searches well.

Technical (all implemented in `RetrievalOptimizer`):

- **Query normalization** (on by default, cheap): fixes typos, expands abbreviations (e.g. "ol" → "Ordinary Level (O/L)"), resolves pronouns using recent chat history, and **translates** the query into the document collection's language when needed (Sinhala/Tamil/ Singlish → English for GIC, since the knowledge base is English) — the *final answer* is still generated in the citizen's original language; only the search step is translated.
- **Query rewriting** (optional, heavier): LLM rewrites a messy query into a clean search query.
- **HyDE** (Hypothetical Document Embeddings, optional): asks the LLM to write a fake "ideal answer" first, then searches for real documents that resemble it — useful when the question and the answer use very different vocabulary.
- **Multi-query / RAG-Fusion** (optional, most expensive): generates several differently-phrased versions of the query, searches with each, and fuses all result sets with RRF — most robust, reserved for cases where accuracy matters more than latency/cost.

3.8 Relevance Filtering & Refusing to Guess

Plain English: if a citizen asks something completely unrelated to GIC, the bot should say "I don't have information about that" instead of forcing an answer out of whatever happens to be the closest (but still irrelevant) document.

Technical: retrieved results are filtered by **L2 distance threshold** (0 = identical meaning, larger = less related); results beyond a configured threshold are dropped, and if even the single best match is beyond a stricter **hard cutoff**, no context is injected into the prompt at all, so the model is explicitly told nothing relevant was found rather than being handed a weak, misleading match.

3.9 Multi-Tenancy

Plain English: the same piece of software runs many completely separate chatbots — one for GIC, one could be for a bank, one for a university — and none of them can see each other's data, exactly like separate tenants in the same apartment building sharing the plumbing but not each other's rooms.

Technical: each tenant is a `Client` row in the SQL database (slug, name, persona, domain template, branding, WhatsApp credentials), paired with its own isolated FAISS collection `client_<slug>` on disk. A `MultiClientRAGPipeline` manager creates/caches one `RAGPipeline` instance per tenant, so different tenants can even run different LLM personas/domains side-by-side under one running backend process. Public API routes are scoped to a single slug with no endpoint that lists or reaches other tenants' data.

3.10 Security

- **Rate limiting & input validation** via a custom `SecurityMiddleware`.
- **CORS** locked to known frontend origins (explicit localhost list in dev; a regex allow-list of the actual hosting providers in production).
- **Trusted-host middleware** in production, preventing Host-header attacks.
- **JWT-based authentication** for the admin console, with three roles: `superadmin`, `operator`, and `client_admin` (scoped to exactly one tenant, signs in at `/portal/<slug>`).
- Public chat endpoints never expose secrets (API keys, WhatsApp tokens) — only branding fields.

3.11 The Learning Loop

Plain English: every conversation is logged, and the system automatically flags the ones that probably went badly — so a human can review and improve the knowledge base over time instead of guessing what's missing.

Technical: every chat turn is written to an `Interaction` row. A turn is auto-flagged `is_weak` if: no knowledge-base match was found, the conversation was escalated to a human, negative emotion was detected, or the citizen gave a 🙄. These feed an admin "Insights" view and an "Escalation Inbox" for human follow-up.

3.12 Data Engineering / Ingestion Pipeline

Plain English: before any of the above can work, someone has to actually go get the organization's real content and turn it into clean, structured records — this is unglamorous but where most real accuracy is won or lost.

Technical (GIC case): gic.gov.lk turned out to be a Next.js single-page application backed by clean, unauthenticated JSON REST APIs (`/api/services`, `/api/organizations`, `/api/forms`, `/api/news`) — discovered by inspecting real network requests rather than scraping rendered HTML, which is far more reliable (no broken selectors, no pagination gaps, byte-exact payloads). A fifth content type — server-rendered static pages like `/about` — genuinely has no API and required actual HTML parsing (BeautifulSoup + `html2text`). Full pipeline detailed in Section 7.

4. Software Requirements Specification

4.1 Scope

The system provides an AI-powered question-answering chatbot over an organization's own knowledge base, deployable per-tenant, reachable via a hosted web chat page, an embeddable JS widget, and WhatsApp; plus an admin console for managing tenants, uploading documents, and reviewing conversation quality.

4.2 User Classes

User Class	Description
Citizen / End User	Asks questions via the hosted chat page, embedded widget, or WhatsApp. No login required.
Client Admin	Owns/manages exactly one tenant (e.g. the GIC deployment): uploads documents, edits persona/branding, reviews escalations. Logs in at a tenant-scoped portal.
Operator / Superadmin	Manages the whole platform: creates tenants, has cross-tenant visibility.

4.3 Functional Requirements

ID	Requirement
FR-1	The system shall accept a natural-language question from a citizen and return a grounded, cited answer.
FR-2	The system shall decide, per message, whether a knowledge-base lookup is needed (agentic behaviour) rather than always forcing retrieval.
FR-3	The system shall cite the specific source page(s)/URL(s) an answer was grounded in.
FR-4	The system shall accept 👍 / 👎 feedback on any answer and persist it against the logged interaction.
FR-5	The system shall support ingesting PDF and JSON-array knowledge sources, chunking and embedding them into a tenant-scoped vector index.
FR-6	The system shall support creating, listing, and configuring independent tenants ("clients"), each with its own persona, branding, and knowledge base.
FR-7	The system shall detect the citizen's message language/script and respond in the same language, regardless of the knowledge base's own language.
FR-8	The system shall support voice input/output (speech-to-text and text-to-speech) as an alternative chat mode.
FR-9	The system shall support receiving and answering messages via WhatsApp Business API per tenant.
FR-10	The system shall log every conversation turn and automatically flag low-quality turns (no match / escalation / negative sentiment / negative feedback) for human review.
FR-11	The system shall allow escalating a conversation to a human agent and record it in an inbox.

FR-12	The system shall refuse to answer (rather than guess) when no sufficiently relevant knowledge-base content exists for a query.
FR-13	The system shall support transactional mock actions (e.g. raising a ticket) initiated conversationally by the agent, recorded and auditable.

4.4 Non-Functional Requirements

Category	Requirement
Accuracy / Trust	Answers must be grounded in retrieved source content; the system must be able to say "I don't know" rather than hallucinate.
Multi-tenancy isolation	One tenant's documents, conversations, and credentials must never be reachable from another tenant's context or public API.
Multilingual support	English, Sinhala, and Tamil must be supported in both query understanding and answer generation.
Resilience	External dependency failures (LLM quota, TTS provider outage) must degrade gracefully (e.g. automatic fallback to a different TTS provider) rather than break the chat.
Re-indexing idempotency	Re-running ingestion on updated source content must fully rebuild the tenant's index without duplicate/stale chunks.
Security	Rate limiting, input validation, CORS/host restrictions, and role-scoped JWT auth for all administrative operations.
Portability	The framework must be domain-agnostic — a new tenant for an unrelated organization must require no code changes, only configuration + data.

4.5 External Interface — Key API Endpoints

Method & Path	Purpose
GET /api/public/{slug}/config	Public branding/config for a tenant's chat page (no secrets).
POST /api/public/{slug}/chat	Send a citizen message, get a grounded answer + sources (the core RAG endpoint).
POST /api/public/{slug}/feedback	Record 👍 / 👎 on a previous answer.
/api/clients/*	Admin: create/list/update tenants.
/api/documents/*	Admin: upload & index knowledge documents per tenant.
/api/query/*	Admin/internal: direct retrieval & query testing.
/api/voice/*	Speech-to-text / text-to-speech voice-call mode.
/api/auth/*	Login, JWT issuance, role-based access.
/webhooks/whatsapp	WhatsApp Business API inbound message webhook.

4.6 Constraints & Assumptions

- The vector index (FAISS "flat" type) performs an exact search and is assumed to operate at a scale of thousands to tens of thousands of chunks per tenant; a larger scale would need an approximate index type (see Section 8 scaling discussion).
- LLM generation depends on a third-party API (Groq or Google Gemini) being reachable and within quota; provider is swappable via one configuration value.
- gic.gov.lk's own JSON APIs are assumed stable in shape; a structural change to those APIs would require updating the fetch/normalize scripts (not the core framework).

5. UML Diagrams

5.1 Use Case Diagram

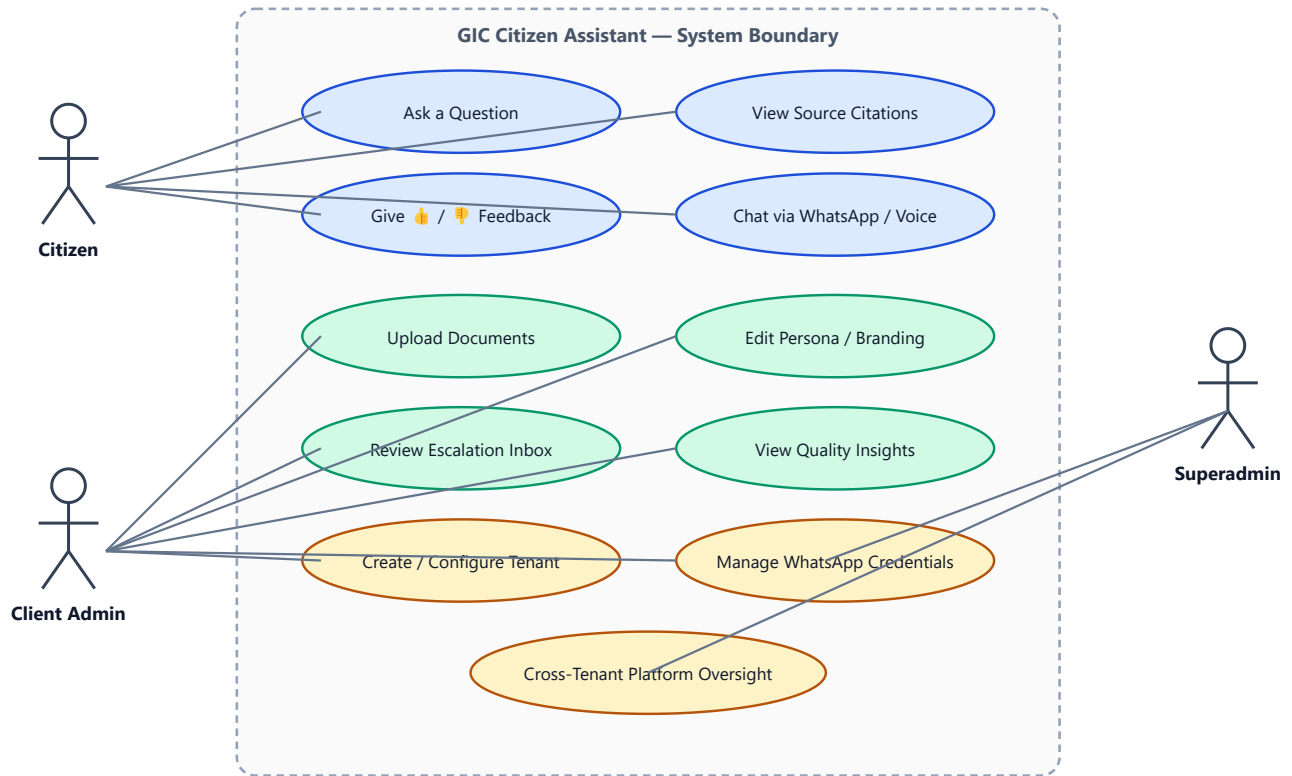
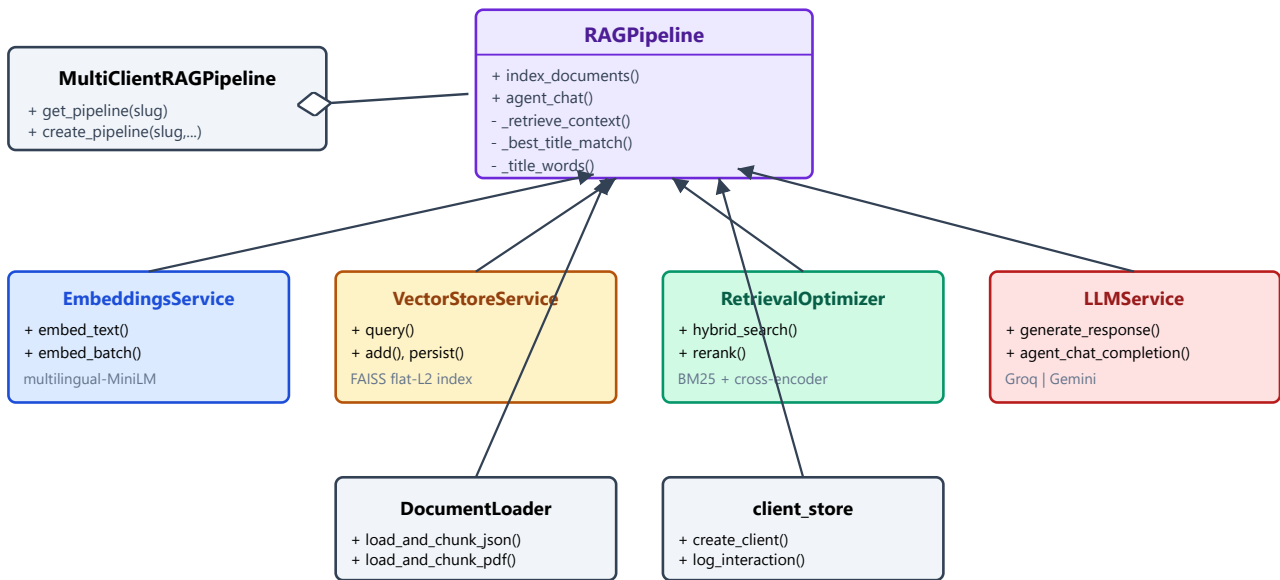


Figure 5.1 — Use Case diagram: actors and their interactions with the system.

5.2 Class Diagram — Service / Engine Layer



RAGPipeline composes one instance of each service; RetrievalOptimizer itself contains a BM25Okapi index and a shared, process-wide CrossEncoder re-ranker cache (loaded once for all tenants).

Solid diamond = composition (RAGPipeline owns/creates these; they don't outlive it)

Open arrow = uses/depends-on relationship

One RAGPipeline instance exists per tenant, cached by MultiClientRAGPipeline

Figure 5.2 — Class diagram of the core RAG engine and its collaborators.

5.3 Sequence Diagram — Answering a Chat Message

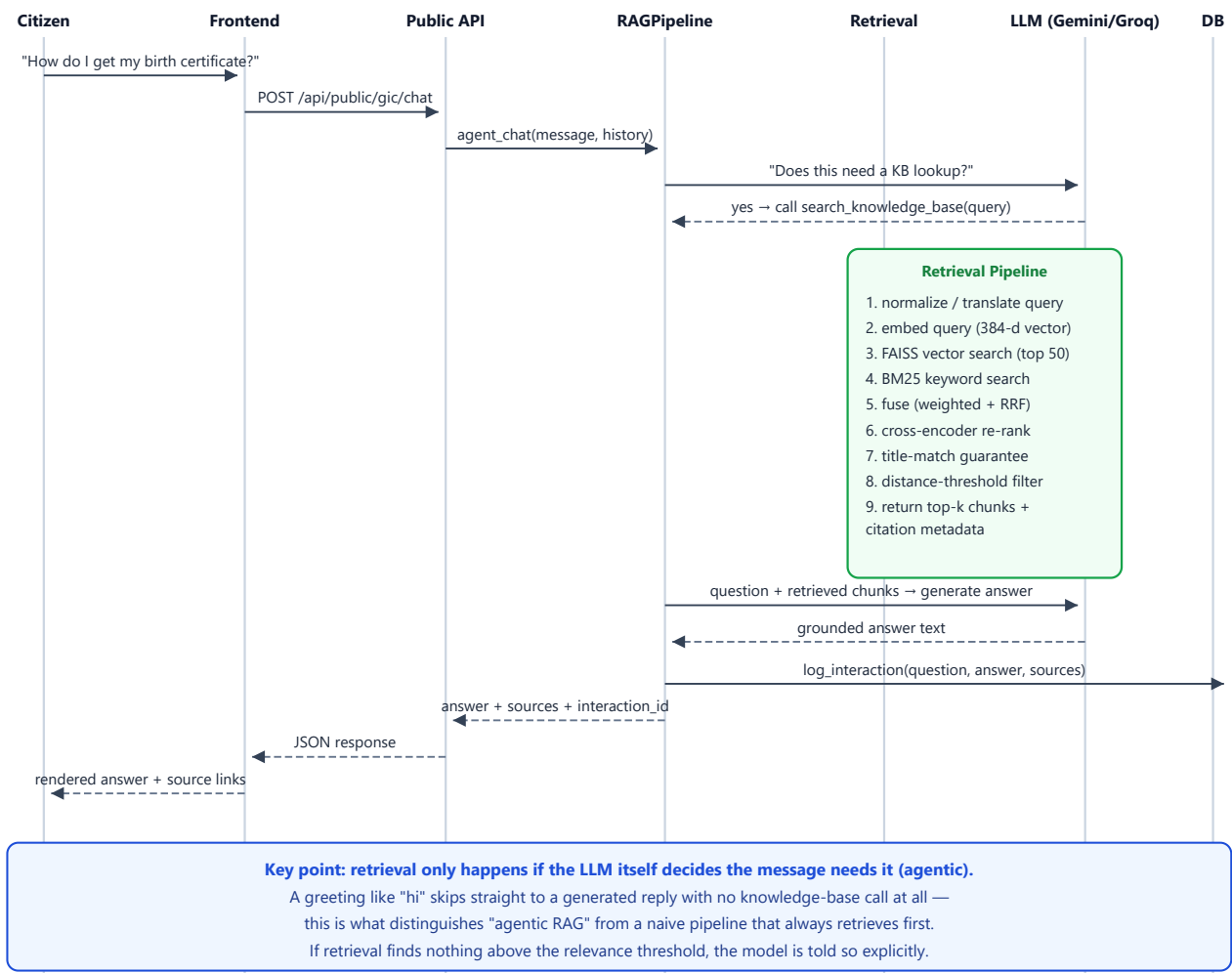


Figure 5.3 — Sequence diagram: end-to-end flow of one chat message.

5.4 Activity Diagram — Knowledge Ingestion Pipeline

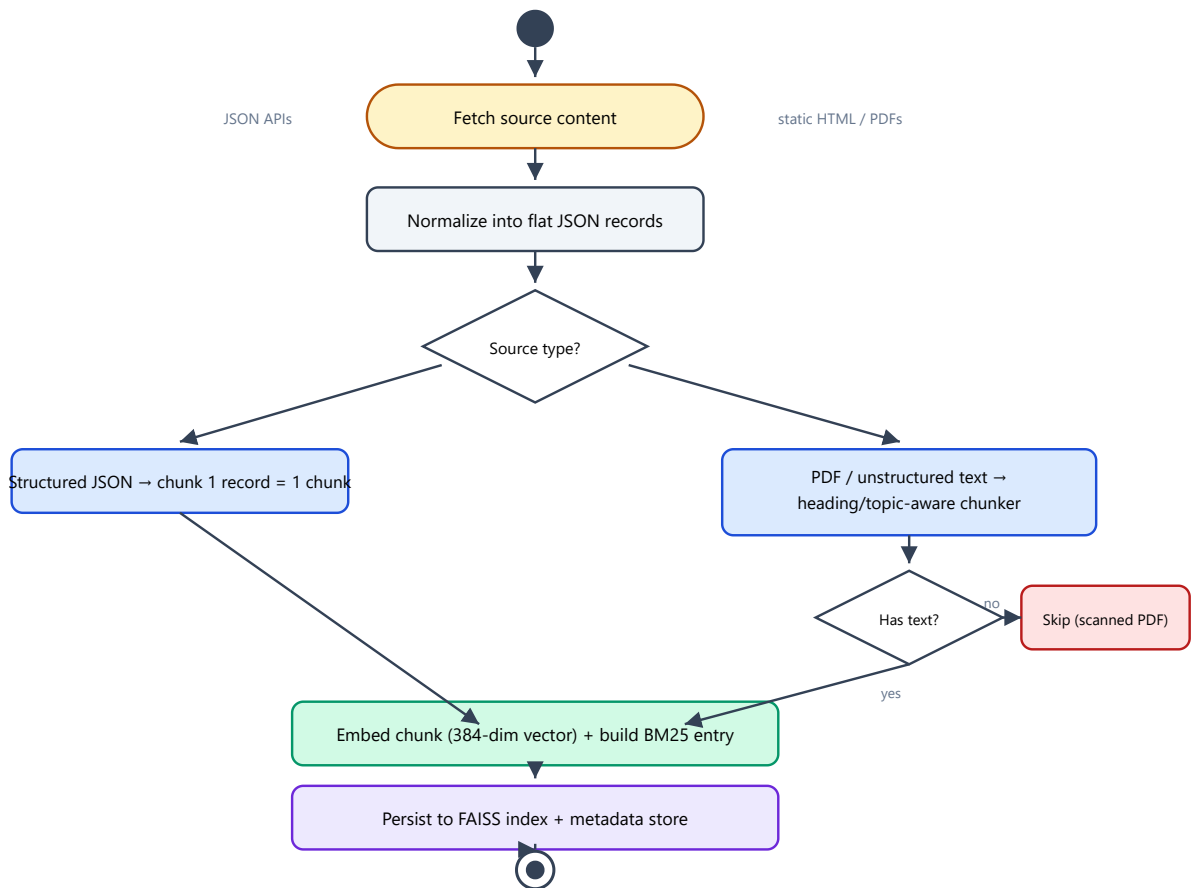


Figure 5.4 — Activity diagram: how raw source content becomes searchable chunks. Re-run per source; re-running fully rebuilds the tenant's index (delete-then-recreate), keeping it idempotent.

5.5 Component Diagram

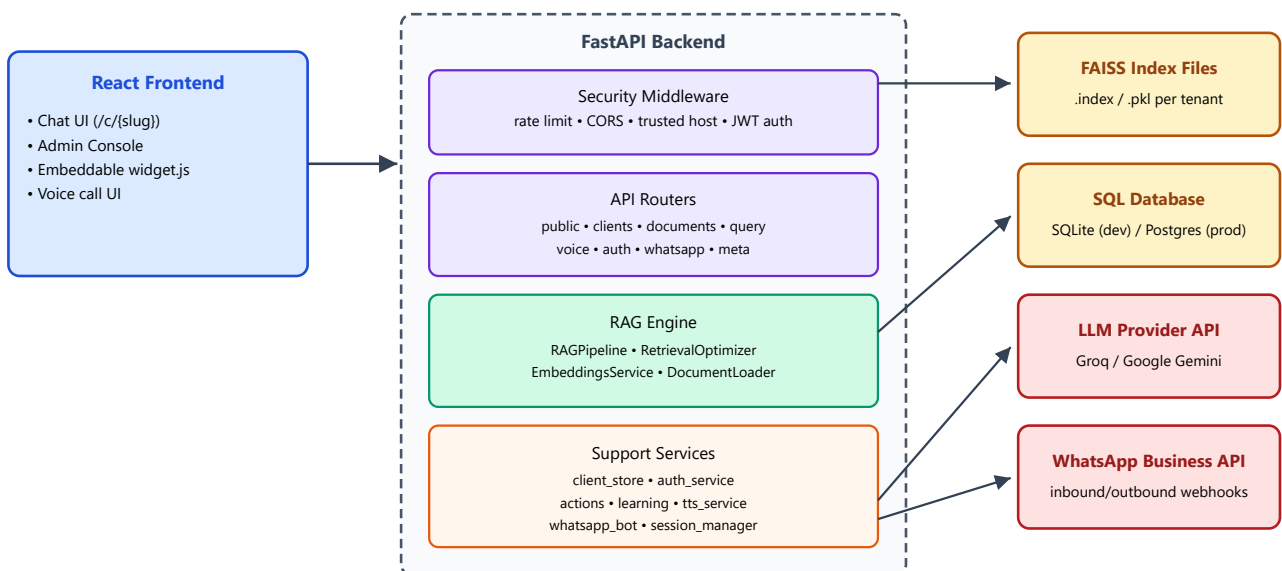


Figure 5.5 — Component diagram: internal module boundaries and external dependencies.

5.6 Deployment Diagram

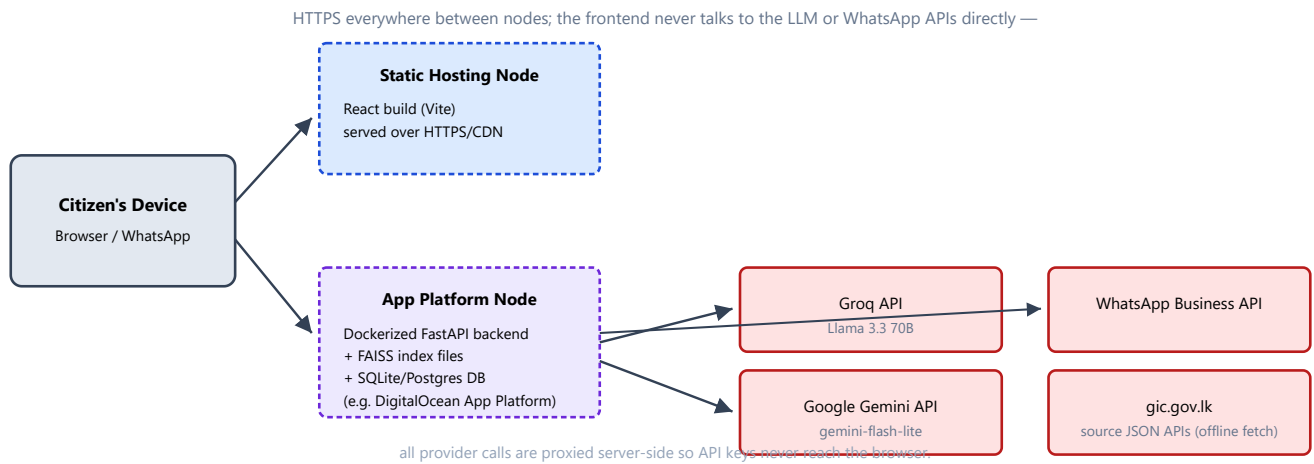
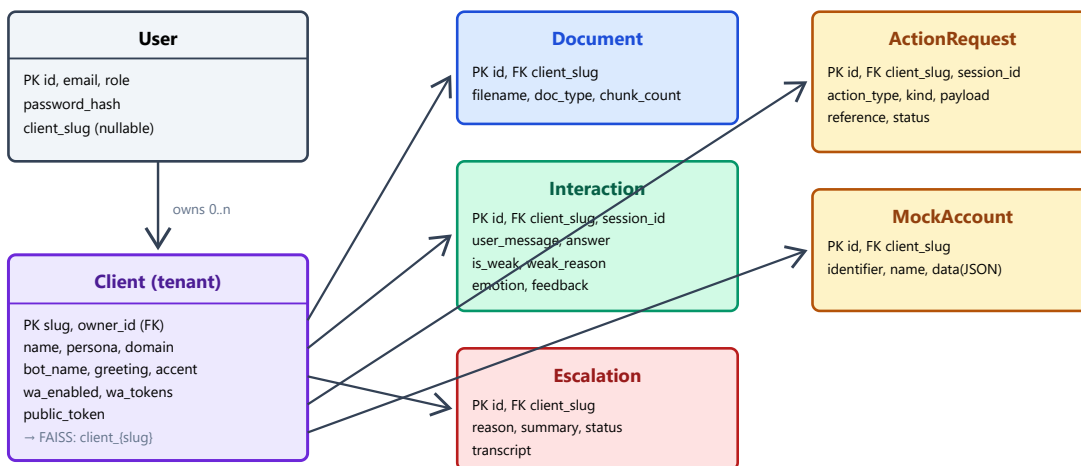


Figure 5.6 — Deployment diagram: physical/hosting topology.

5.7 Entity–Relationship Diagram — Database Schema



Every operational table is scoped by client_slug — this IS the multi-tenant isolation boundary at the DB level.
Vector data lives outside the SQL DB entirely, in one FAISS collection file-pair per tenant.

Figure 5.7 — Entity–Relationship diagram of the relational schema.

6. End-to-End System Flow — Narrated

This section walks through exactly what happens for one real example, pairing plain English with the technical step, so you can narrate it fluently in an interview.

Citizen types: "How do I get a copy of my birth certificate?"

Step 1 — Transport: the React chat page sends this as JSON to `POST /api/public/gic/chat`, along with a `session_id` (so the backend can remember earlier turns in this conversation) and the recent message history.

Step 2 — Tenant resolution: the backend looks up the `gic` tenant, confirms it exists and has a working LLM configured, and hands off to that tenant's cached `RAGPipeline` instance.

Step 3 — Agent decision: the LLM is called with the message, the conversation history, and one available tool (`search_knowledge_base`). Because this is clearly a factual question about a GIC service, the model calls the tool with a search query.

Step 4 — Query understanding: the query is normalized (typo/abbreviation cleanup, translated to English if it arrived in Sinhala/Tamil/Singlish).

Step 5 — Retrieval: the normalized query is embedded into a 384-dimension vector; FAISS returns the nearest chunks by L2 distance; BM25 independently returns its own keyword-ranked chunks; both lists are fused (weighted score + Reciprocal Rank Fusion).

Step 6 — Precision pass: the fused short-list is re-scored by the cross-encoder re-ranker (reads query+chunk together); separately, any chunk whose own *title* strongly matches the query is guaranteed a place in the final set, even if its body text scored lower — this is what makes "Obtain a Certified Copy of the Certificate of Birth" win over a longer, superficially similar page about registering a *new* birth.

Step 7 — Relevance gate: if the best match is still too distant (off-topic question), no context is passed at all and the model is told nothing relevant was found.

Step 8 — Generation: the LLM (Gemini in this deployment) is prompted with the tenant's persona + the retrieved chunks + the original question, and told to answer *only* from what was retrieved, in the citizen's original language.

Step 9 — Logging & response: the turn is written to the `Interaction` table (for the learning loop), and the answer plus a list of source titles/URLs is returned to the frontend and rendered with clickable citations.

7. Case Study — Building the GIC Knowledge Base

This section is worth knowing well: it demonstrates applied, not just theoretical, understanding, and interviewers often ask "walk me through something real you built."

7.1 Discovering the Real Data Source

Rather than assuming `gic.gov.lk` needed to be scraped page-by-page (slow, brittle, breaks on redesign), the site's actual network traffic was inspected first. It turned out `gic.gov.lk` is a Next.js single-page application backed by four clean, unauthenticated JSON REST APIs:

Source	Shape	Ingested volume
<code>/api/services</code>	12 categories → 78 sub-services → 763 leaf items	763 items + overview records per level
<code>/api/organizations</code>	19 top-level orgs → 669 sub-organizations	~688 org + sub-org records
<code>/api/forms</code>	13 categories → 754 service links	96 actual downloadable PDF forms
<code>/api/news</code>	flat array	6 articles
Static SSR pages	<code>/about</code> , homepage sections	parsed directly from real server-rendered HTML (not from <code><main></code> , which only wraps a floating chat widget on this site — verified by reading the raw HTML, not assumed)

Every nesting level (category / sub-service / leaf item, and top-level org / sub-org) was given its own precise citation URL matching the real page it renders on — a citizen question about a sub-organization cites that sub-organization's own page, not its parent's.

7.2 Two Real Bugs Found and Fixed

Bug 1 — Hybrid search was silently merging unrelated documents

The vector-search half of hybrid search was labelling each result with its *rank position within that one query's results* (0, 1, 2, ...) instead of its true, stable position in the collection — while BM25 used real collection-wide positions. When the two result lists were merged by that ID, unrelated documents got matched up by coincidence of rank rather than identity, corrupting which chunks survived into the final answer. **Fix:** the vector store's `query()` now also returns each match's true FAISS index position, and all three call sites that merge hybrid results were updated to use it. Verified by re-running a previously-inconsistent question multiple times and confirming stable, correct sourcing.

Bug 2 — A precisely-labelled record was losing to a longer near-duplicate

"How do I get a copy of my birth certificate?" should surface the record titled "*Obtain a Certified Copy of the Certificate of Birth*" — but it was being crowded out by a longer page about a related-but-different process, because the cross-encoder re-ranker scores whole document bodies, and a short precisely-titled record can lose that contest to a long keyword-dense one. **Fix:** Section 3.6's title-match guarantee — a retrieval-level

rule that trusts the source's own label, not a prompt-level "please be careful" instruction. This was a deliberate design choice: fixing it at the retrieval layer means the *correct* source gets cited, not just a hedge added around a wrong one.

7.3 Other Practical Fixes Along the Way

- **Language-model choice:** Groq's Llama 3.3 70B occasionally produced broken, repeating-character output on Sinhala questions; switched the deployment's default provider to Google Gemini, which is markedly more fluent in Sinhala — a one-line configuration change thanks to the provider being abstracted behind one interface.
- **Graceful skip of scanned PDFs:** a handful of form PDFs are scanned images with no extractable text; the ingestion script now checks for this and skips them with a warning instead of crashing the whole run.
- **Idempotent re-ingestion:** because FAISS has no real "delete," each ingestion run rebuilds the tenant's collection from a fresh process rather than appending, so re-running after a GIC content update never produces duplicate or stale chunks.

Final result: **1,861 chunks** indexed into the `client_gic` FAISS collection, covering services, organizations, forms, news, and static pages, deployed on DigitalOcean App Platform with the frontend pointed at the backend's public URL.

8. Interview Q&A Cheat Sheet

Q: What is RAG, and why not just fine-tune the model on your data?

A: RAG retrieves relevant real documents at answer time and grounds the model's response in them, instead of baking facts into model weights. Fine-tuning is expensive to redo every time content changes, doesn't reliably teach exact facts (models still hallucinate specifics learned via fine-tuning), and gives no citation trail. RAG updates instantly when you re-index new documents, and every answer can point back to its source.

Q: Why FAISS instead of Pinecone, Weaviate, or Chroma?

A: FAISS runs embedded in-process with no external service to operate, which fits a self-hosted, per-tenant deployment model well — each tenant is just a pair of files on disk. At larger scale or if you needed managed infrastructure/horizontal scaling, a hosted vector DB would be the natural next step; the abstraction (`VectorStoreService`) is already isolated behind an interface so that swap wouldn't touch the rest of the pipeline.

Q: What's the difference between vector search and keyword search, and why use both?

A: Vector search compares *meaning* (via embeddings) so it survives paraphrasing and synonyms, but can miss exact terms like codes or names. Keyword search (BM25) is exact-term-precise but blind to paraphrasing. Hybrid search runs both and fuses the results, covering each other's blind spots.

Q: What is re-ranking, and why not just use the top vector search results directly?

A: The first retrieval pass (bi-encoder embeddings) is fast because query and documents are encoded independently and compared by simple distance — but that independence is also its weakness. A cross-encoder reads the query and each candidate document *together* and produces a much more accurate relevance score, at the cost of being too slow to run against the whole collection — so it's only applied to the short-list that survives the fast first pass.

Q: How do you stop the model from hallucinating?

A: Two layers: (1) the prompt instructs the model to answer only from the retrieved context, and (2) a distance-based relevance gate withholds context entirely if nothing in the knowledge base is actually close enough to the question, so the model is told explicitly that nothing relevant was found rather than being handed a weak match to improvise around.

Q: How is multi-tenancy / data isolation implemented?

A: Every tenant is a row in a `Client` table plus its own FAISS collection file pair named `client_<slug>`. Every operational table (interactions, escalations, documents, actions) carries a `client_slug` column. Public API routes are scoped to one slug per request with no endpoint that lists or crosses into other tenants' data.

Q: How would this scale to millions of documents?

A: The current FAISS index is a "flat" index — exact but linear-time search. At millions of vectors you'd switch to an approximate nearest-neighbor index type (e.g. HNSW or IVF+PQ), which trades a small amount of recall for large speed gains, and/or move to a horizontally-shardable managed vector database. The re-ranking step would also need a tighter candidate cap, since cross-encoder scoring is the most expensive step per query.

Q: What does "agentic" mean in this system specifically?

A: The LLM is given a tool (`search_knowledge_base`) and decides for itself, per message, whether to call it — rather than the code always forcing a retrieval step before generation. A greeting doesn't trigger a knowledge-base search; a factual question does. This is implemented via LangChain's tool-calling / function-calling interface.

Q: How is multilingual support handled without a separate model per language?

A: One multilingual embedding model covers English, Sinhala, and Tamil in a shared vector space. Since the knowledge base itself is stored in English, incoming queries in Sinhala/Tamil/romanized "Singlish" are first translated to English purely for the search step (so they match the English-embedded documents well) — but the final answer is generated back in the citizen's original language, so the translation is invisible to the user.

Q: What was the hardest bug you dealt with, and how did you find it?

A: A silent data-corruption bug in hybrid search fusion, where vector results were keyed by their rank position instead of a stable document ID, causing the fusion step to accidentally merge unrelated documents. It didn't crash anything — it just quietly degraded answer quality. Found it by reproducing a wrong answer consistently across repeated identical queries, then testing retrieval in isolation to prove the correct document *was* retrievable, which narrowed the bug down to the fusion/merge step rather than retrieval itself.

Q: What tradeoffs did you make, and why?

A: Reranking and multi-stage query transformation add real latency and LLM call cost, so the system enables cheap steps (query normalization, hybrid search, reranking) by default but leaves the most expensive ones (multi-query fusion, HyDE) off by default, reserved for cases that need maximum recall. Chunk size and top-k are tuned to balance context completeness against prompt size/cost.

Q: Why build a generic multi-tenant framework instead of a one-off GIC bot?

A: The retrieval/generation core (embeddings, vector store, hybrid search, reranking, agentic chat) has nothing GIC-specific about it — only the ingested documents and persona are domain-specific. Building it as a reusable framework meant a second, unrelated organization's chatbot needs zero new code, only new data and configuration.

9. Glossary of Terms

Term	Meaning
RAG	Retrieval-Augmented Generation — search for relevant real content first, then ask a language model to answer using it.
Embedding	A fixed-length list of numbers representing the meaning of a piece of text, positioned so that similar meanings end up close together.
Vector store / vector database	A database specialized for storing embeddings and finding the nearest ones to a query vector quickly.
FAISS	Facebook AI Similarity Search — an open-source library for fast vector similarity search, used here as the vector store.
L2 distance	Euclidean distance between two vectors; smaller = more semantically similar in this system.
Chunk	A small, self-contained piece of a document, sized to be one focused unit of retrievable meaning.
BM25	A classic statistical keyword-ranking algorithm (the same family used by many search engines) used here as the keyword half of hybrid search.
Hybrid search	Combining vector (semantic) search and keyword (BM25) search results into one ranked list.
Reciprocal Rank Fusion (RRF)	A formula for merging multiple ranked lists by rewarding documents that rank highly in any of them, without needing comparable score scales.
Cross-encoder	A model that scores a query and a document <i>together</i> in one pass — slower but more accurate than comparing separately-computed embeddings; used for re-ranking.
Bi-encoder	A model that encodes the query and each document independently (as separate embeddings) — fast enough to run against a whole collection, used for the first-pass search.
Re-ranking	A second, more precise scoring pass applied only to the short-list from the first (fast) retrieval pass.
HyDE	Hypothetical Document Embeddings — generating a fake "ideal answer" first and searching for real documents that resemble it, to bridge vocabulary gaps between questions and answers.
Agentic / tool-calling	The language model itself decides when to invoke a tool (like a knowledge-base search) rather than the code always forcing that step.
Multi-tenancy	One running system serving many isolated customers ("tenants"), each with separate data and configuration.
Hallucination	A language model confidently generating plausible-sounding but false or unsupported information.
LLM	Large Language Model — the generative AI model that writes the final answer text (e.g. Gemini, Llama).

JWT	JSON Web Token — a signed token used to authenticate admin/API requests without a server-side session store.
SRS	Software Requirements Specification — a structured document describing what a system must do (functional) and how well it must do it (non-functional).

End of document — GIC Citizen Assistant, SRS & System Design, v1.0